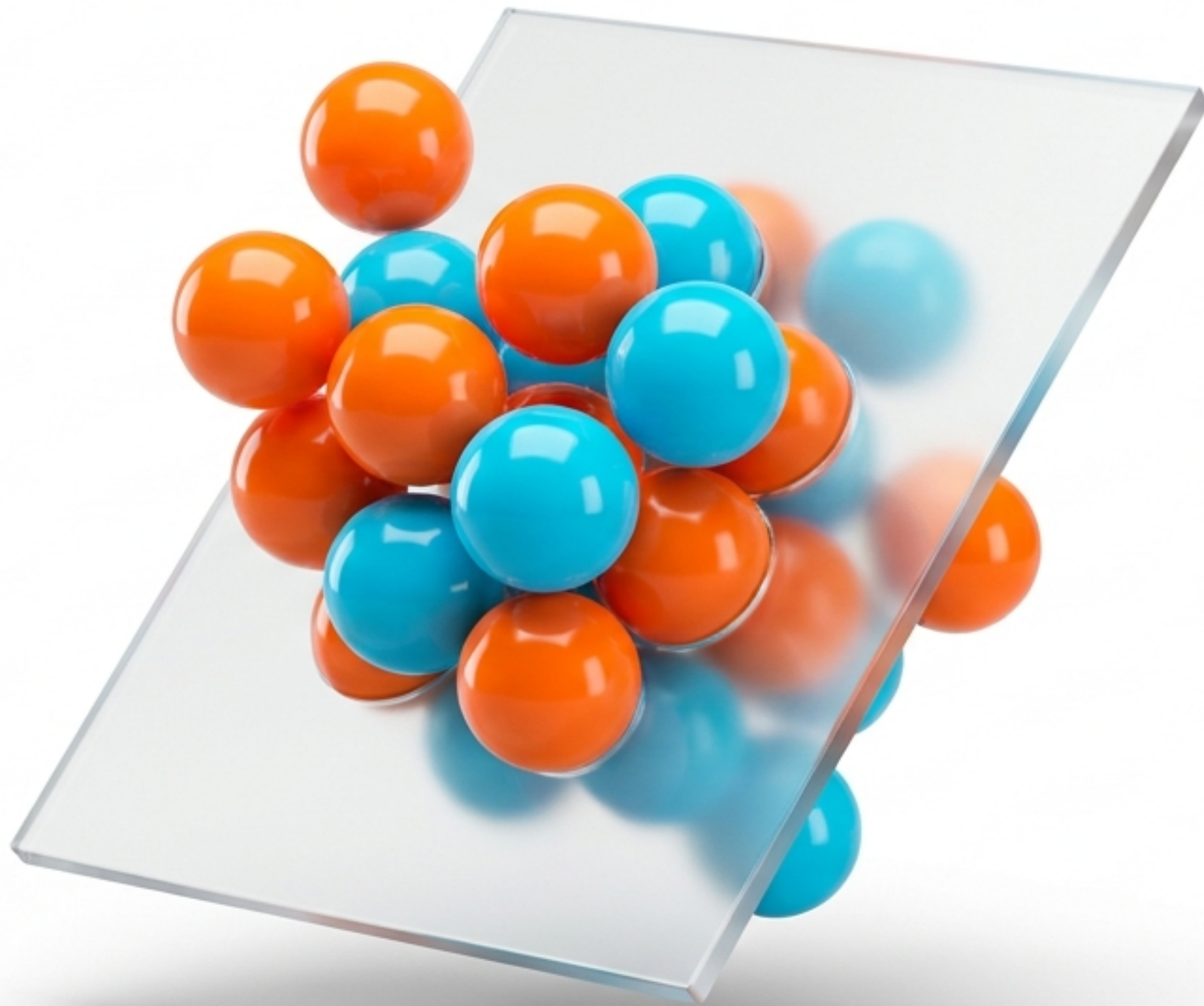


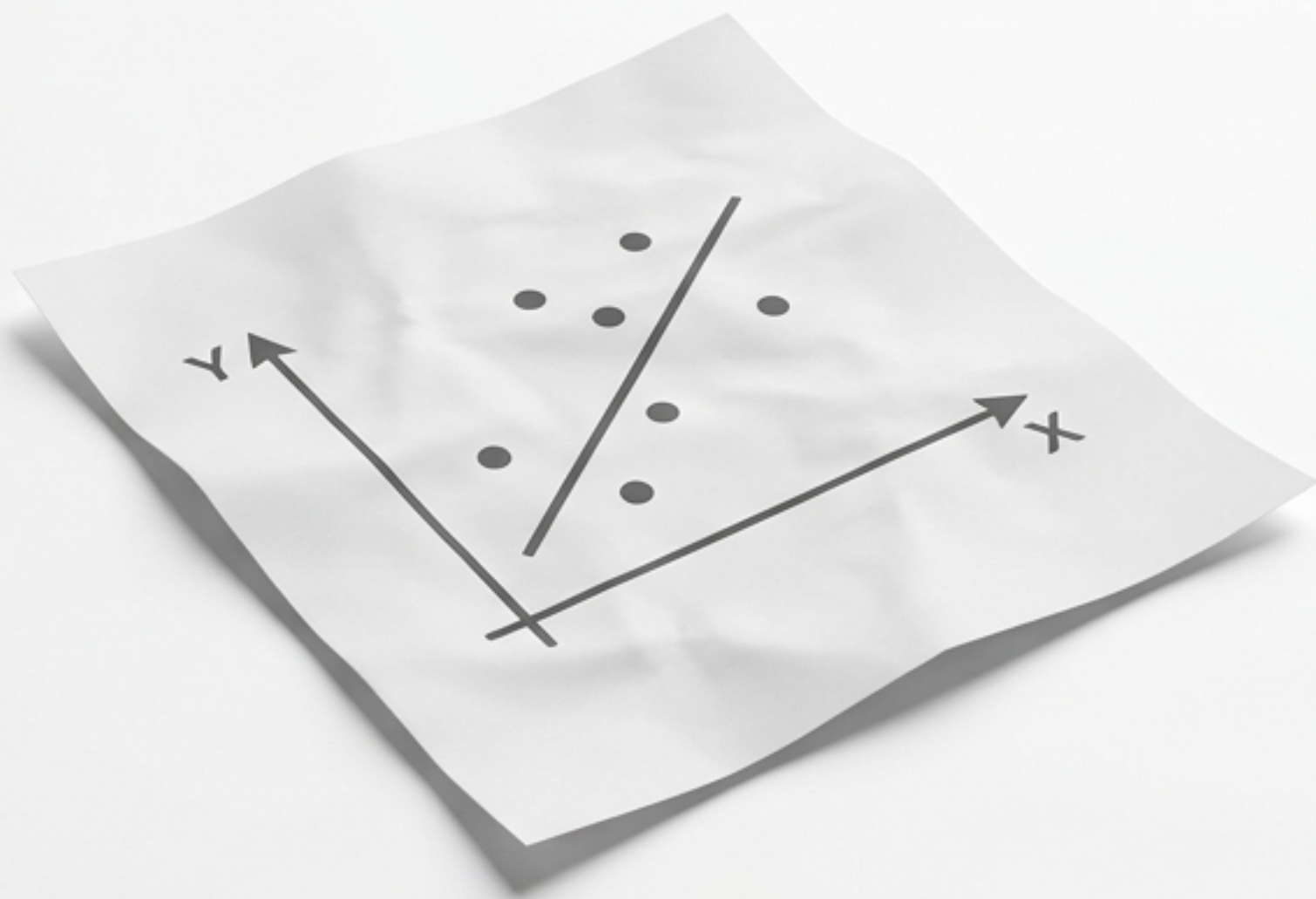
Giải Mã Cơ Chế **Siêu Phẳng SVM**

Từ ảo ảnh 2D đến sức mạnh thực sự
trong không gian n-chiều.



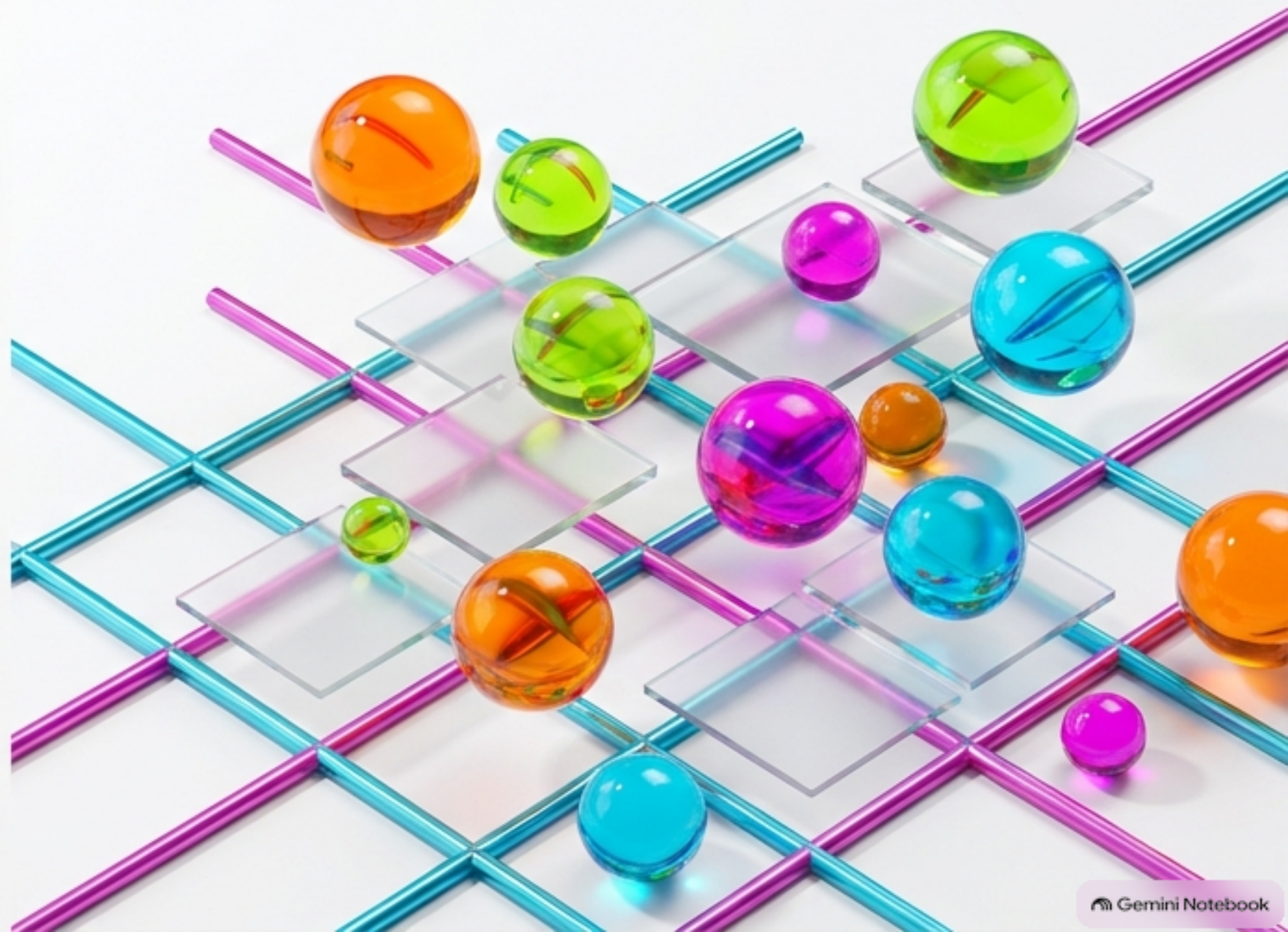
Làm tương 2D

- Sách vở thường chỉ vẽ 2 thuộc tính (x_1 , x_2).
- **Nguyên nhân:** Giới hạn hiển thị của mặt phẳng giấy và màn hình.



Thực tế n-chiều

- Thuật toán **SVM** dễ dàng xử lý hàng trăm đến hàng ngàn thuộc tính cùng lúc (x_1 , x_2 , ..., x_n).



Hệ số chệch (Bias)

- Dịch chuyển **ranh giới quyết định** song song.



$$s(x) = \mathbf{w}^T \mathbf{x} + b = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$



Trọng số (Weights)

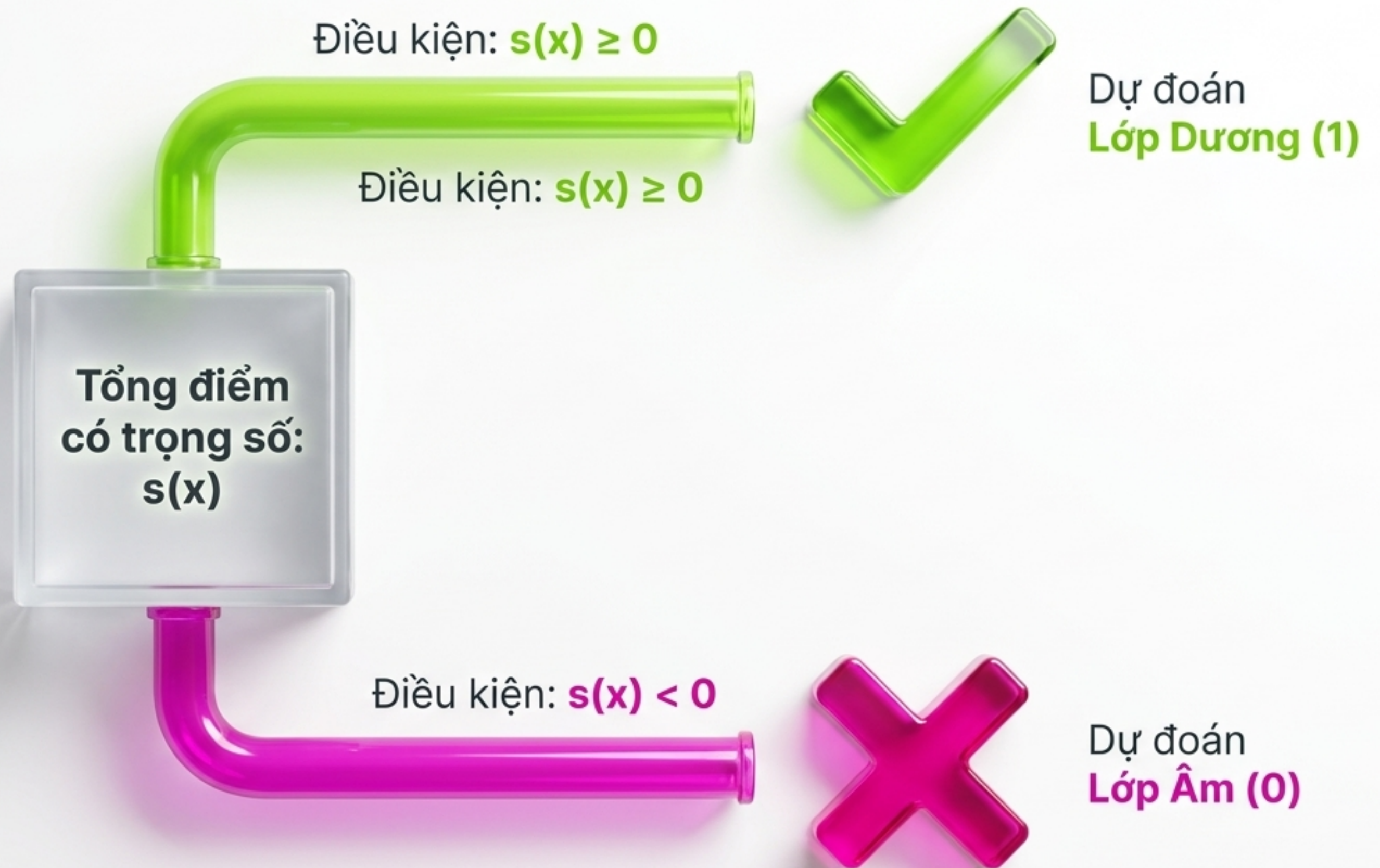
- Mức độ quan trọng của từng thuộc tính **w1 ... wn**.
- Trực tiếp định hình độ nhạy và hướng trục của **siêu phẳng**.



Thuộc tính (Features)

- Các giá trị đầu vào **x1 ... xn**.







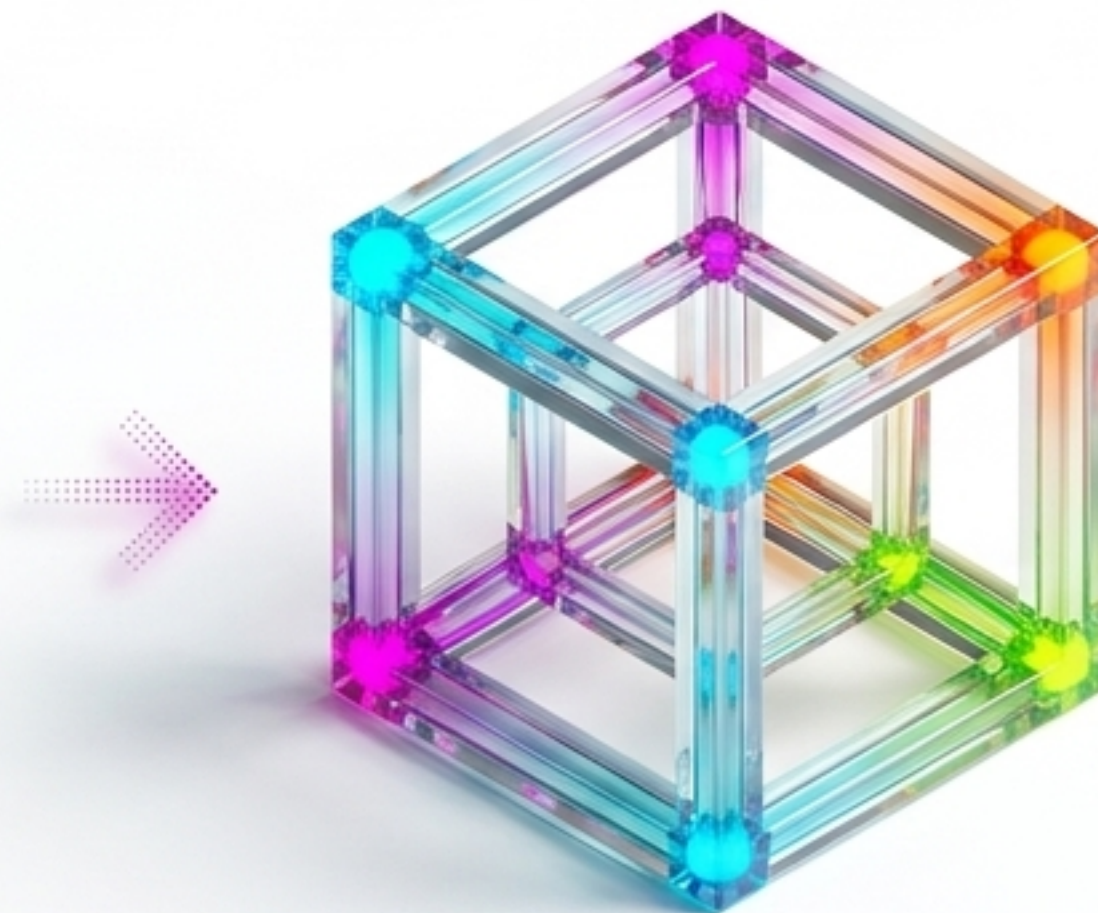
2 Thuộc tính (x_1, x_2)

Hình dạng: Đường thẳng 2D



3 Thuộc tính (x_1, x_2, x_3)

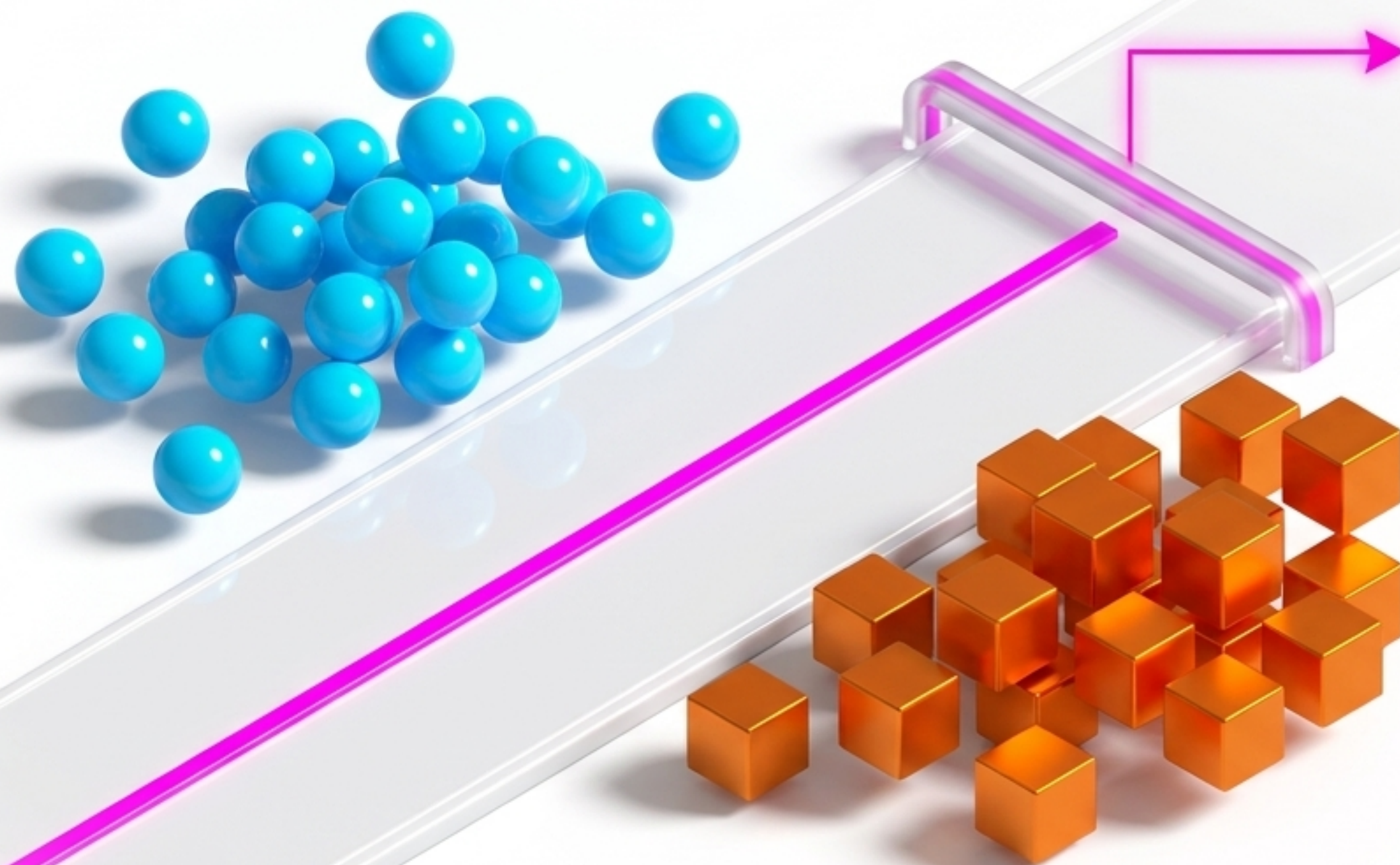
Hình dạng: Mặt phẳng 3D



n Thuộc tính ($n > 3$)

Hình dạng: Siêu phẳng (Hyperplane)
trong không gian n -chiều

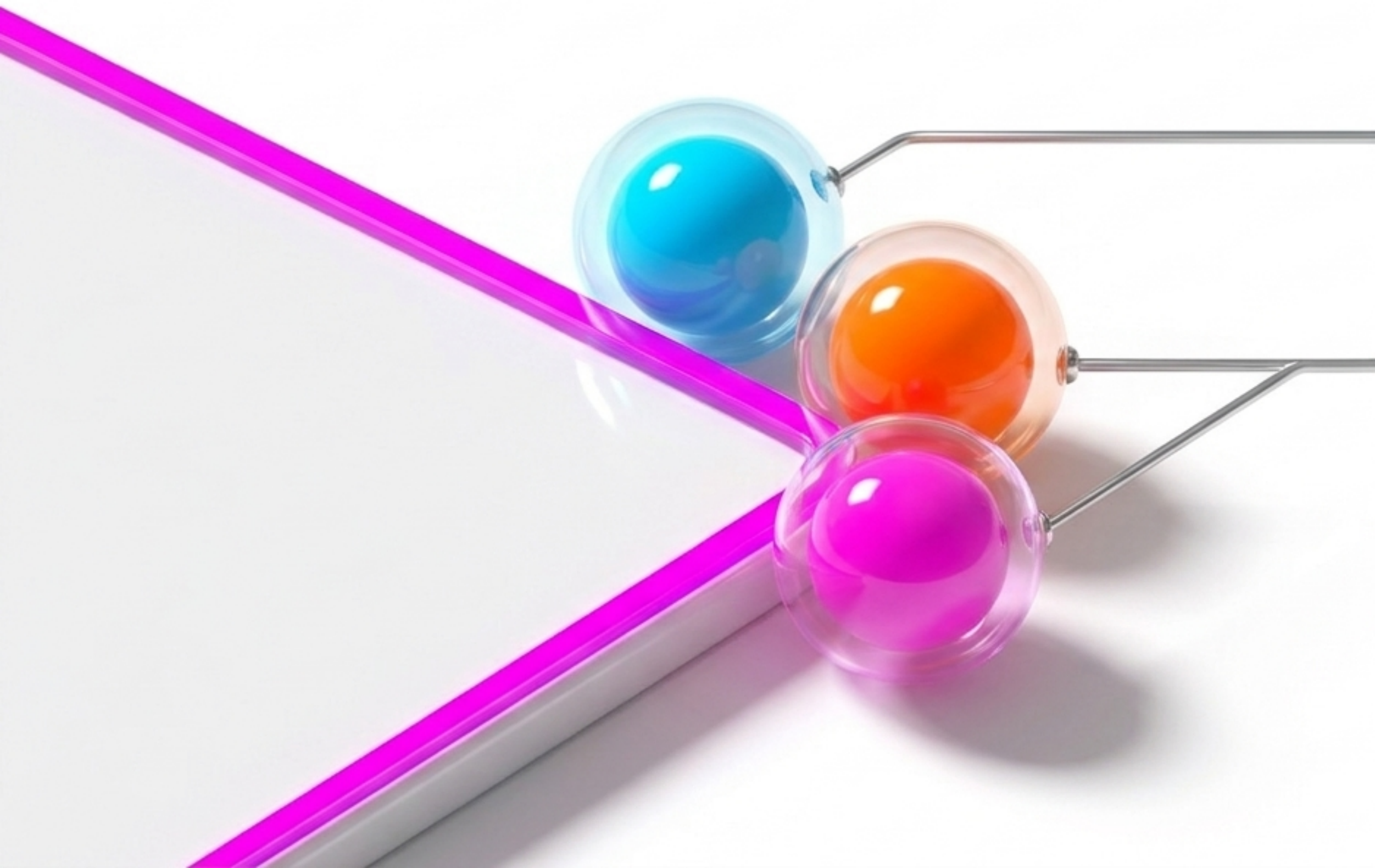
Mục tiêu tối thượng của SVM không bao giờ thay đổi.



Tối đa hóa Lề (Margin)

- Dù ở không gian bao nhiêu chiều, thuật toán luôn tìm kiếm **Siêu phẳng** tạo ra **con đường phân cách rộng nhất** giữa các lớp dữ liệu.

Mỏ neo của Siêu phẳng: Vector Hỗ Trợ



Định vị ranh giới:

- Nằm ngay trên mép của con đường.

Quyền lực tuyệt đối:

- Vị trí và độ nhạy hướng trục của **siêu phẳng** bị chi phối hoàn toàn bởi tọa độ của những điểm này. Các điểm dữ liệu khác không hề **ảnh hưởng**.

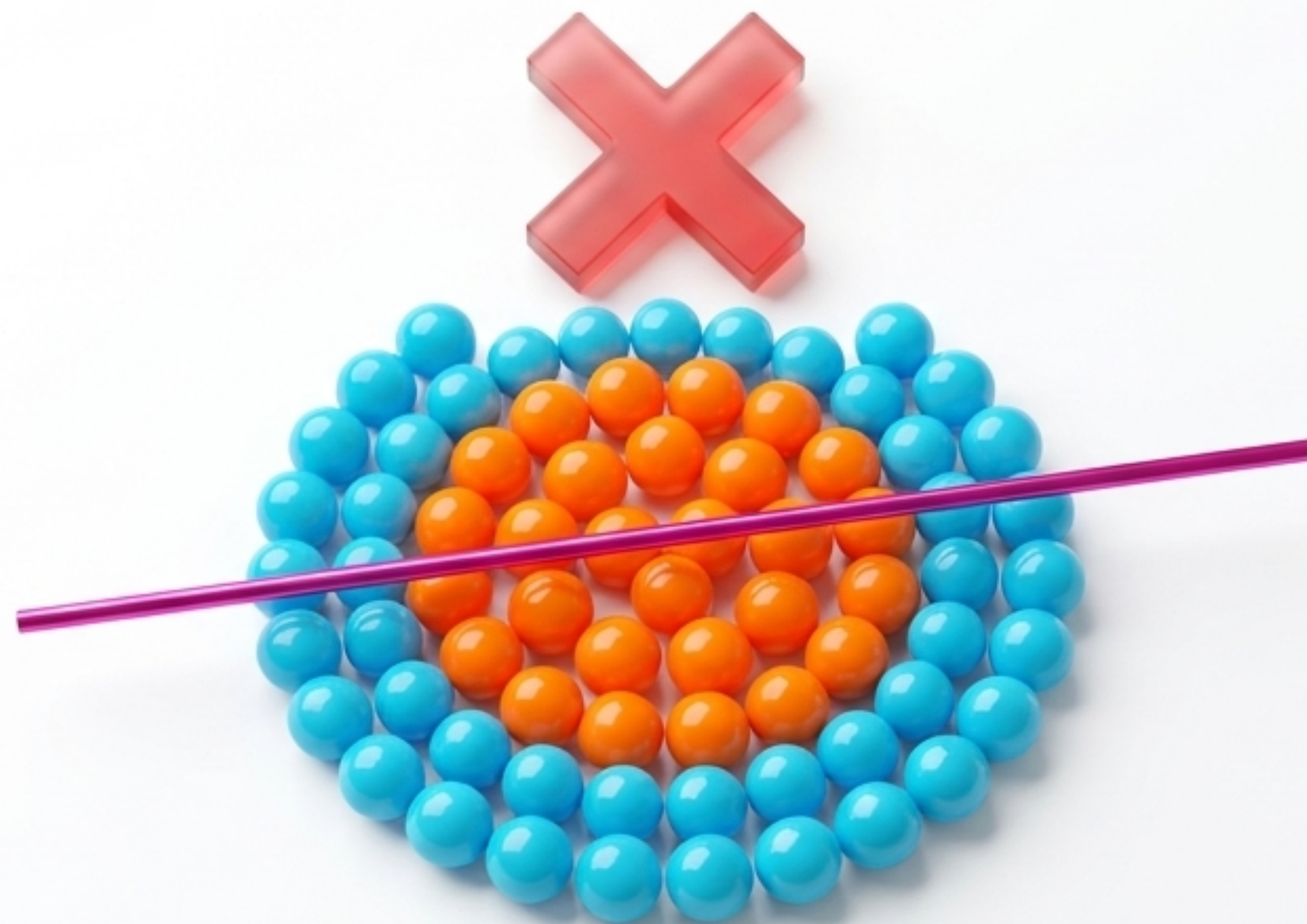
Bản chất thực sự của một Vector Hỗ Trợ

Không phải là một
giá trị đơn lẻ!

Mỗi vector là một mẫu dữ liệu hoàn chỉnh,
chứa tọa độ đầy đủ của n thuộc tính:

$$\mathbf{x}^{(i)} = (x_1^{(i)}, x_2^{(i)}, \dots, x_n^{(i)})$$

Thách thức: Dữ liệu Phi tuyến



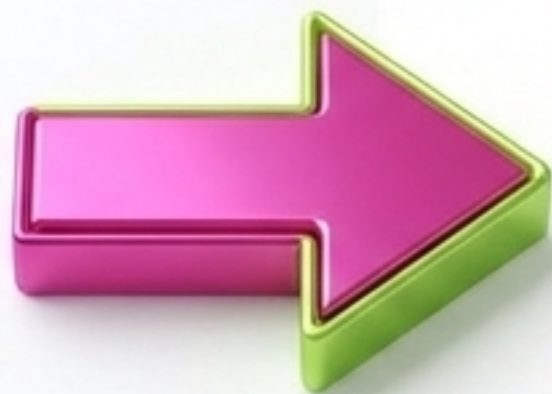
Trong không gian gốc, dữ liệu đan xen phức tạp. Không thể dùng bất kỳ một **siêu phẳng** tuyến tính (đường thẳng/mặt phẳng) nào để phân tách chúng một cách hoàn hảo.

Giải pháp 1: Thêm đặc trưng đa thức thủ công



Từ (x_1, x_2)

Tạo thêm (x_1^2, x_2^2, x_1x_2)



Kết quả: Dữ liệu cuộn lại thành không gian 3D, nơi một mặt phẳng có thể dễ dàng cắt ngang để phân tách.

Giải pháp 2: Phép thuật Kernel (Kernel Trick)

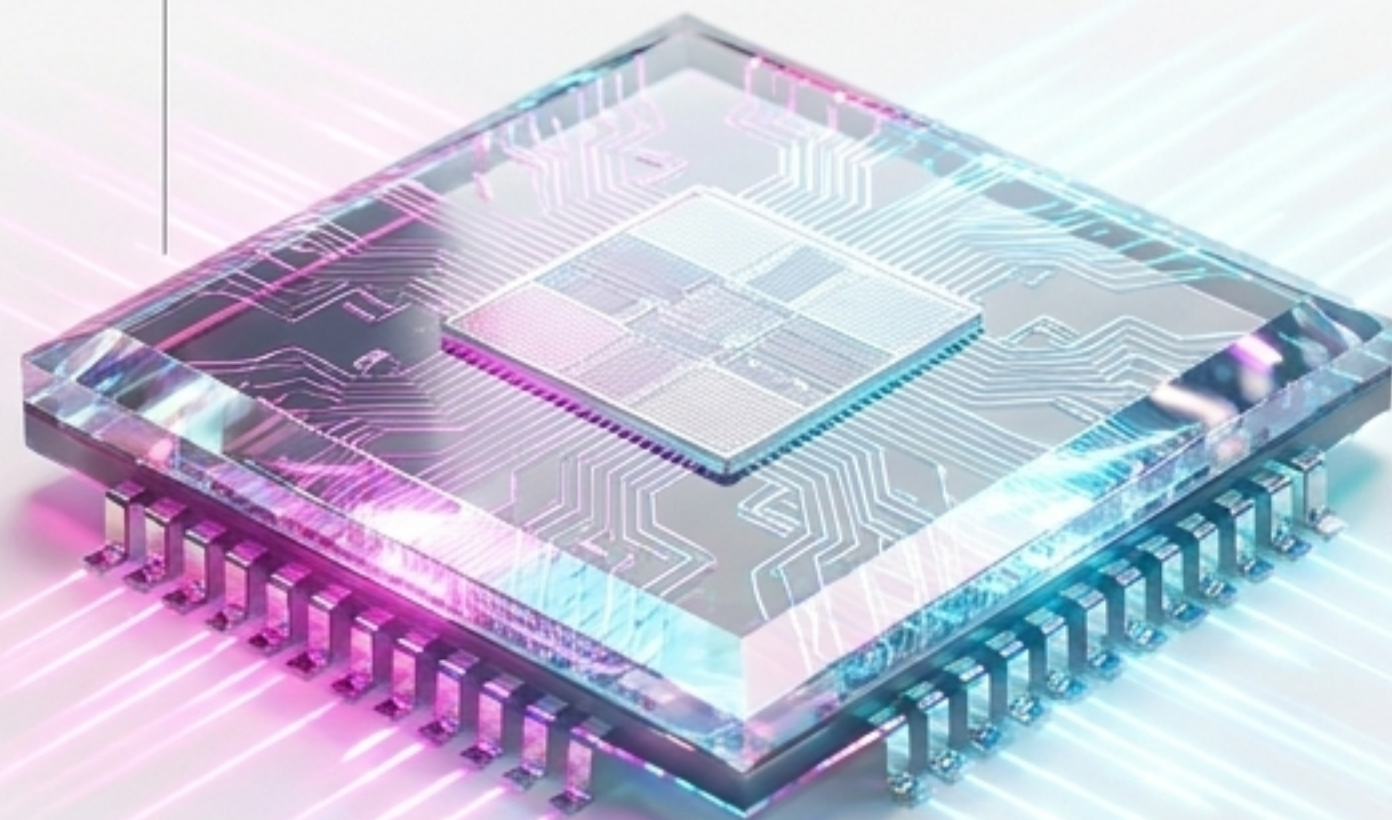


● Vấn đề:

- Thêm hàng ngàn thuộc tính thủ công sẽ làm cạn kiệt bộ nhớ máy tính.

● Phép thuật Kernel:

- Tính toán trực tiếp điểm số trong không gian nhiều chiều mà không thực sự tạo ra hay lưu trữ các thuộc tính mới trong bộ nhớ.
- Ví dụ: Kernel Đa thức, Kernel Gaussian (RBF).



Phương pháp	Bản chất Không gian	Chi phí Bộ nhớ	Tốc độ
Thêm đặc trưng thủ công	Tăng số chiều thực sự	Rất Cao 	Chậm 
Kernel Trick	Chiều ảo (Toán học)	Tối ưu 	Cực Nhanh 

Tổng kết: Vượt khỏi mặt phẳng giấy



- Hình vẽ x_1, x_2 chỉ là công cụ minh họa trực quan.

- Thực tế, **SVM** đưa tất cả thuộc tính $x_1 \dots x_n$ vào phương trình đại số để tìm **siêu phẳng phân tách tối ưu**.

Bạn đã sẵn sàng để hiểu cách **SVM** xử lý khi số lượng thuộc tính còn lớn hơn cả số lượng mẫu dữ liệu ($n > m$)?